ðŸ’Ž ShopSense

Smart Credit Tracking & Retail Billing System

Object-Oriented Programming (OOP) Open-Ended Lab Project

DEVELOPMENT TEAM MEMBERS

Ryan Nasir (Student ID: 74832) - Core OOP Architect

Muhammad Umar (Student ID: 74786) - Backend & Persistence

Muhammad Wahaj (Student ID: 74742) - UI Layout & Styling

Muhammad Asim Abbasi (Student ID: 74844) - Ledger & Analytics

INSTRUCTOR: Faiza Latif Abbasi

SEMESTER: Spring 2026

SLOT: 08:30 to 11:20 - WEDNESDAY

Software Development Documentation: ShopSense Smart Credit Tracking & Retail Billing System

1. Project Overview & Current Contents

ShopSense is a desktop Java application designed to digitize manual credit registers ("udhaar khata") for small retail businesses. The system manages customer records, logs credit and payment transactions, dynamically calculates balances, alerts shopkeepers when credit limits are breached, exports formatted text invoices, and generates charts for business reporting.

What the Project Currently Contains:

- **Core Entities:** Java classes implementing abstraction, inheritance, interfaces, and encapsulated data.
- **Persistence Layer:** A file handling module utilizing native Java I/O to read and write records from local text files.
- **Custom Exception Framework:** Custom exceptions that safeguard the business rules.
- **JavaFX User Interface:** A styled graphical dashboard implementing sidebar navigation, metrics panels, tables, transactional forms, and progress-bar analytics.

---

2. Existing Screens & UI Navigation Flow

The interface consists of a **Sidebar Navigation Menu** on the left and a **Main Content Area** on the right. The screens are:

1. **Dashboard Home:**

- Displays four Metric Cards: *Total Outstanding Balance*, *Total Credit Extended*, *Total Payments Received*, and *Active Credit Limit Breaches*.
- Includes a *High-Risk Alerts Table* listing customers who have exceeded their limits, with a quick-access button to record repayments.

2. **Customer List:**

- Displays all registered customers in a tabular format (ID, Name, Phone, Email, Outstanding Balance, Credit Limit, and Status).
- Features a *Real-Time Search Bar* allowing the shopkeeper to search instantly.

- Contains CRUD action buttons: *Add New Customer*, *Edit*, and *Delete*.

3. **Udhaar Ledger:**

- Features a *Customer Selection Dropdown* with autocomplete.
- Displays a *Customer Account Profile Card* containing name, ID, outstanding balance, credit limit, and healthy/breach status.
- Includes action controls: *Log Credit (Udhaar) Purchase* and *Record Cash Payment*.
- Displays a *Transaction History Ledger Register* table listing all transaction logs with a button to *Export Invoice*.

4. **Analytics & Reports:**

- Displays a **Top 5 Debtors Bar Chart** showing the customers with the largest outstanding balances.
- Displays an **Outstanding Credit Category Breakdown** showing credit distribution by categories: *Groceries*, *Medicines*, *Household*, and *Others*.

---

3. Implemented Core Functionalities

- **Customer CRUD:** Full Create, Read, Update, and Delete operations.
- **Real-Time Linear Search:** Fast linear lookup filtering matching records as the user types.
- **Overloaded Transaction Logging:** Allows adding credit (categorized) and payment transactions using overloaded signatures.
- **Polymorphic Balance Calculation:** Outstanding balance is computed dynamically by traversing transactions and summing their polymorphic effects (`+amount` for credit, `-amount` for payment).
- **Credit Limit Safeguard with Exception Handling:** Adding credit checks if the simulated balance exceeds the customer's limit. If so, a `CreditLimitExceededException` is thrown, which the UI catches to display an override dialog.
- **Print Receipt / Invoice Export:** Generates a formatted text invoice file saved under a `receipts/` directory.

---

4. Directory & Project Structure

The project directory must be structured as follows:

```
```

ShopSense/

â”,

â”œâ”€â”€ javafx-sdk-21.0.2/ # Standalone JavaFX SDK library folder

â”, â”"â”€â”€ lib/ # JavaFX jar files (controls, graphics, base, etc.)

â”,

â”œâ”€â”€ src/ # Java source code root folder

â”, â”"â”€â”€ com/

â”, â”"â”€â”€ shopsense/

â”, â”,

â”, â”œâ”€â”€ Main.java # Application entrypoint launcher

â”, â”,

â”, â”œâ”€â”€ ShopSenseApp.java # Main Ul layout and event controllers

â”, â”,

â”, â”œâ”€â”€ ShopSenseTest.java # Backend verification test suite

â”, â”,

â”, â”œâ”€â”€ model/ # Domain entity classes and interfaces

â”, â”, â”œâ”€â”€ Person.java

â”, â”, â”œâ”€â”€ Customer.java

â”, â”, â”œâ”€â”€ Alertable.java

â”, â”, â”œâ”€â”€ Transaction.java

â”, â”, â”œâ”€â”€ CreditTransaction.java

â”, â”, â”œâ”€â”€ PaymentTransaction.java

â”, â”, â”œâ”€â”€ Account.java

â”, â”, â”"â”€â”€ CustomerAccount.java

â”, â”,

â”, â”œâ”€â”€ exception/ # Custom Exception definitions

```
â”, â”, â”œâ”€â”€ CustomerNotFoundException.java

â”, â”, â”"â”€â”€ CreditLimitExceededException.java

â”, â”,

â”, â”œâ”€â”€ util/ # Storage utilities

â”, â”, â”"â”€â”€ StorageManager.java

â”, â”,

â”, â”"â”€â”€ ui/ # Styling sheets

â”, â”"â”€â”€ styles.css # Design system styles

â”,

â”œâ”€â”€ bin/ # Compiled Java .class bytecode output

â”œâ”€â”€ data/ # Persistent database flat-files

â”, â”œâ”€â”€ customers.txt

â”, â”"â”€â”€ transactions.txt

â”,

â”"â”€â”€ receipts/ # Formatted text receipts directory
```

---

5. OOP Concepts Implementation Details

- **Encapsulation:** All attributes in `Person`, `Customer`, `Transaction`, and `Account` are `private` or `protected`. State is accessed and modified strictly via public getters/setters.
- **Inheritance:**
- `Customer` extends `Person` (inheriting demographic fields).
- `CreditTransaction` and `PaymentTransaction` extend `Transaction`.
- `CustomerAccount` extends `Account`.
- **Abstraction:**
- `Account` is an abstract class defining `calculateBalance()` and `addTransaction()`.
- `Transaction` is an abstract class defining `getEffectOnBalance()` and `getTransactionType()`.
- **Polymorphism (Overriding):**

- Subclasses of `Transaction` override `getEffectOnBalance()` (`CreditTransaction` returns `+amount`, `PaymentTransaction` returns `-amount`).
- Subclasses of `Transaction` override `toString()`.
- `CustomerAccount` overrides `calculateBalance()` to aggregate the ledger.
- **Polymorphism (Overloading):** `CustomerAccount` contains overloaded methods:
- `addTransaction(Transaction)`
- `addTransaction(amount, notes, type)`
- `addTransaction(amount, notes, type, date)`
- `addTransaction(amount, notes, type, date, category)`
- **Interfaces:** `Alertable` interface defines `checkCreditLimit()` and `sendAlert()`. It is implemented by `Customer` to enforce the credit-check warning contract.

---

6. Setup & Eclipse Configuration Guide (Step-by-Step)

To build and run this exact project in Eclipse, follow these steps:

Step 1: Create a Java Project in Eclipse

1. Open Eclipse, click **File > New > Java Project**.

2. Name the project `ShopSense`. Ensure you select **JavaSE-21** (or compatible JDK) and click **Finish**.

3. Right-click the `src` folder, select **New > Package**, and name it `com.shopsense`. Create additional subpackages: `com.shopsense.model`, `com.shopsense.exception`, `com.shopsense.util`, and `com.shopsense.ui`.

Step 2: Configure the Standalone JavaFX SDK

1. Download **JavaFX Windows SDK 21.0.2** zip file from Gluonhq.

2. Unzip it and place the `javafx-sdk-21.0.2` folder directly in your Eclipse workspace `ShopSense` project root folder.

3. In Eclipse, right-click the `ShopSense` project and select **Properties**.

4. Go to **Java Build Path** on the left menu, select the **Libraries** tab.

5. Click **Modulepath** (or **Classpath** depending on version), then click **Add External JARs...**.

6. Navigate into your project directory `javafx-sdk-21.0.2/lib/`, select all `.jar` files (e.g. `javafx.base.jar`, `javafx.controls.jar`, etc.), and click **Open**.

7. Click **Apply and Close**.

Step 3: Configure Eclipse VM Arguments to Run JavaFX

To launch the application without modular warnings in Eclipse:

1. Copy all `.java` code files into their respective folders under `src/com/shopsense/`.

2. Right-click `Main.java` and choose **Run As > Run Configurations...**.

3. Under the **Arguments** tab, find the **VM Arguments** text area and input the following configurations:

```
--module-path "${project_loc}/javafx-sdk-21.0.2/lib" --add-modules javafx.controls
```

4. Click **Apply**, then click **Run**.

---

7. Step-by-Step Module Implementation Plan

Phase 1: Initialize Core OOP Entities (Assigned to: Ryan Nasir)

1. **`Person.java`:** Write attributes (`id`, `name`, `phoneNumber`, `email`) and getters/setters.

2. **`Transaction.java`:** Create abstract definition.

3. **`CreditTransaction.java` & `PaymentTransaction.java`:** Extend `Transaction`. Override `getEffectOnBalance()` to return `+amount` and `-amount`. Add category field for credit.

4. **`Account.java`:** Define abstract class with transaction list.

5. **`CustomerAccount.java`:** Implement balance traversal loop. Set up the three overloaded `addTransaction` signatures.

6. **`Alertable.java`:** Declare interface methods.

7. **`Customer.java`:** Extend `Person`, implement `Alertable`. Link it to `CustomerAccount`.

Phase 2: Persistence & Error Safeguards (Assigned to: Muhammad Umar)

1. **Custom Exceptions:** Implement `CustomerNotFoundException` and `CreditLimitExceededException`.

2. **`StorageManager.java`:** Implement file handling. Set up `saveData(ArrayList)` and `loadData()` using pipe-separated parsing. Connect relational customer-transaction links.

3. **`ShopSenseTest.java`:** Create a text-only test file to assert balance math, limit checks, exception triggers, and file saving/loading. Compile and run it from the console to verify backend correctness.

Phase 3: Graphical Layout & Design Tokens (Assigned to: Muhammad Wahaj)

1. **`styles.css`:** Write the custom styling theme using the exact color tokens (`#7AB342`, `#FFFFFF`, `#1E2A14`, `#C8D9B5`, etc.) and font pairings (`Poppins` headings, `Inter` body, `JetBrains Mono` prices).

2. **Sidebar Panel:** Construct left sidebar layout using a VBox. Link button actions to main StackPane selectors. Add the Open-Ended Lab slot and student metadata at the footer.

3. **Dashboard Panel:** Construct metric cards and breach tables. Implement mouse event listeners for card hover scaling.

Phase 4: Ledger Controls, Invoice Systems, & Analytics (Assigned to: M. Asim Abbasi)

1. **Ledger Controls:** Create customer selectors, transaction forms, and ledger table. Hook buttons to dialog modals.

2. **Limit Override flow:** Catch `CreditLimitExceededException` during credit purchase logs and prompt the user with confirmation dialogs.

3. **Invoice System:** Code the receipt exporter under `receipts/` using formatted text writers.

4. **Analytics Panel:** Construct the Top 5 Debtors progress bars and the category-wise breakdown layouts.

---

8. Team Task Distribution Matrix

Team Member	Student ID	Primary Modules / Responsibilities	Deliverables
Ryan Nasir (Lead)	74832	Core OOP Domain Classes & Architecture Design	Domain Entities package (Person, Customer, Account, CustomerAccount, Alertable, Transaction, Credit/Payment Transaction)
Muhammad Umar	74786	Backend Logic, Custom Exceptions, and File I/O Persistence	Exception Classes, StorageManager, text flat-files setup, and JUnit/Verification Test Suites
Muhammad Wahaj	74742	UI Frontend Layout Design, Stylesheet and Navigation	styles.css, Sidebar Menu, Dashboard Screen, Customer Directory CRUD panels
Muhammad Asim Abbasi	74844	Ledger Modules, Invoice Generation, & Reports Analytics	Transaction Forms, Credit override dialogs, Receipt file print utility, Top 5 Debtors list, and category charts

---

9. System Workflows

Frontend-Backend Workflow

- **Application Startup:** GUI requests `StorageManager.loadData()`. Storage reads raw records from text files, instantiates `Customer` and `Transaction` subclasses, builds the ledger, and returns the lists to populate tables.
- **Transaction Processing:** Log buttons trigger calculations. If a transaction causes a breach, an exception is thrown and caught to launch confirmation dialogs.
- **Real-Time Auto-Save:** Adding or editing records automatically calls `StorageManager.saveData()`, updating flat-files instantly.

UI Flow & Screen Navigation

- **Sidebar Selection:** Left sidebar buttons control visibility inside a right-hand layout `StackPane`.
- **Inter-view Navigation:** Clicking "Add Repayment" in the Dashboard breaches table or "View Ledger" in the Customer Directory redirects the user to the Udhaar Ledger view with the selected customer pre-loaded.

---

10. Integration & Assembly Plan

Once each team member completes their assigned files, they will integrate their code into the final project structure as follows:

1. **Step 1: Core Integration**

Integrate the domain classes (`com.shopsense.model.*`) and exception classes (`com.shopsense.exception.*`) created by **Ryan Nasir** and **Muhammad Umar**.

2. **Step 2: Persistence Integration**

Bind the `StorageManager` created by **Muhammad Umar** to save state on app shut-down or when adding/editing customers and transactions. Run `ShopSenseTest` to verify integration correctness.

3. **Step 3: Frontend Styling & Layout Assembly**

Load the stylesheet `styles.css` created by **Muhammad Wahaj** and instantiate the main JavaFX Application panels (`Sidebar`, `Dashboard`, `Customer Table`).

4. **Step 4: Functional Wiring**

Integrate the transaction ledger views, limit alert modals, and reporting charts coded by **Muhammad Asim Abbasi** into the main `ShopSenseApp.java` file.

5. **Step 5: Final Validation**

Compile and run the fully integrated project to confirm:

- Alternating rows and selection highlights render correctly in our green-and-white theme.
- Text elements are visible and clear.
- Forms validate numeric inputs correctly.
- Receipt invoices save accurately in the `receipts/` directory.